

Tech Stack, Architecture & Implementation Plan

AI-powered real-time monitoring and prediction platform for the North Eastern Region

10-Day Hackathon Build | September 2026

01 Architecture in One Line

React frontend talks to a FastAPI backend. The backend loads a pre-trained scikit-learn model, serves risk scores and GeoJSON, stores field reports in SQLite, and fires SMS alerts via Twilio. Everything is deployed on free-tier cloud services. That is the whole system.

Repository Structure

```
project/  
  backend/      # FastAPI app + ML model + mock data  
  frontend/     # React + Vite + Leaflet  
  data/         # GeoJSON, CSVs, rasters
```

```
notebooks/ # Training + feature engineering
README.md
```

02 Backend

FastAPI (Python 3.11)

FastAPI is the right call because the ML stack is Python, so the model lives in the same process as the API. No serialization overhead, no inter-service calls, no separate inference server. The model is loaded once at startup via **joblib** and **model.predict()** is called directly inside route handlers. Automatic Swagger docs at **/docs** double as a testing interface during development.

SQLite via SQLAlchemy

Field reports and alert logs are the only persistent data. SQLite handles both in a single file with zero configuration. SQLAlchemy provides an ORM so raw SQL is never needed. Spatial data (district boundaries, risk zones) lives as static GeoJSON files on disk, loaded at startup and held in memory.

APScheduler

A background scheduler runs the risk scoring function every 15 minutes in-process. It pulls the latest simulated sensor readings, runs batch inference on all district centroids, updates an in-memory dict with current risk levels, and checks which districts have crossed alert thresholds. No Celery, no Redis, no queue infrastructure.

API Endpoints

Method	Route	Description
GET	/risk/zones	NER district GeoJSON enriched with current risk levels
GET	/risk/predict	On-demand inference for lat/lon + rainfall + moisture params
GET	/sensors/live	48-hour simulated sensor time-series for a station ID
GET	/alerts/active	Districts currently at High or Critical risk
POST	/alerts/send-sms	Trigger Twilio SMS for given district and risk level
POST	/reports	Accept geo-tagged photo + description from field officers
GET	/reports	Return all submitted field reports with photo URLs

03 Machine Learning

Model: scikit-learn Random Forest Classifier

Three output classes: **Low, High, Critical**. Trained offline in a Jupyter notebook, serialized with **joblib**, committed to the repository. Training happens once before the hackathon. Inference is a single **predict_proba()** call that completes in under 5 milliseconds.

Random Forest is chosen for three reasons: it handles mixed feature types (numerical slope alongside categorical land cover) without preprocessing gymnastics; it provides **feature_importances_** out of the box for the presentation; and it is robust to class imbalance via **class_weight='balanced'**.

Feature Set

Feature	Source / Method
Elevation	SRTM lookup at coordinate via elevation Python library
Slope Angle	Computed from 3x3 DEM neighborhood using numpy gradient
Land Cover Class	ESA WorldCover raster point-sampled with rasterio
Month of Year	Seasonality signal -- monsoon months get higher prior
Rainfall (mm)	Live from IMD API at inference; distribution-generated for training
Soil Moisture (%)	Simulated at inference; distribution-generated for training

Training Data

Positive samples come from the NASA Global Landslide Catalog (~12,000 events globally, filtered to India). Negative samples are randomly generated coordinates inside NER boundaries with no recorded event within 5 km. Class imbalance is handled with **class_weight='balanced'** and SMOTE on the training set.

Key design decision: `rainfall_mm` and `soil_moisture_pct` are synthetic for training, generated from distributions conditioned on class label. The model learns that high values correlate with positive class. At inference time you feed real/simulated live values and the risk score responds correctly. This is the trick that makes the demo feel genuinely dynamic.

04 Data Layer

Public Datasets Used

Dataset	Source	Used For
NER District Boundaries	GADM (gadm.org)	GeoJSON polygons for map and spatial joins
Global Landslide Catalog	NASA EarthData	Positive training samples (~12k events)
SRTM 30m DEM	USGS EarthExplorer	Elevation + slope angle features
ESA WorldCover 10m	ESA Viewer (free)	Land cover class feature per coordinate
IMD District Rainfall	IMD Public JSON API	Live rainfall value injected at inference time

Simulated Sensor Data

A **mock_data.py** module generates realistic 48-hour time-series for named stations using numpy. Rainfall and moisture are correlated, have a diurnal pattern, and two stations always show a danger spike in the last 12 hours. This spike is the demo trigger: the scheduler picks it up, flips those districts to Critical, and the map recolors within 30 seconds.

IMD Integration

IMD's public JSON endpoint is polled once per scheduler cycle. The rainfall value for NER districts is extracted and fed into the inference pipeline alongside simulated moisture values. This gives one real external data source to point to during the presentation. If the endpoint is unreachable on demo day, fallback to simulated values is silent.

05 Frontend

React + Vite + TypeScript

Vite because the dev server starts in under a second. TypeScript because GeoJSON feature objects and API response shapes are passed around extensively. No Redux, no Zustand. **React Query** handles all server state: caching, background refetching, and loading states. Components never call `fetch()` directly.

React-Leaflet for the Map

The main map renders NER district GeoJSON as a styled layer where fill color maps to risk level (green/orange/red). Each district polygon has an `onClick` handler that sets selected district state, which drives the sidebar. Custom markers for field reports use Leaflet's **Divicon**, allowing any HTML as a marker including photo thumbnails. Leaflet is chosen over MapLibre GL for its documentation depth and Stack Overflow coverage -- both matter when you are debugging at 2am.

Recharts for Charts

The sidebar shows a 48-hour LineChart of rainfall and soil moisture for the selected district, fetched from `/sensors/live`. Recharts integrates cleanly with React state and looks presentable without custom styling.

React Query Polling

`useQuery` with a `refetchInterval` of 30 seconds on `/risk/zones` and `/alerts/active`. When the scheduler updates a district to Critical, within 30 seconds the map recolors it. For a demo, this is indistinguishable from true real-time.

i18next Multilingual Support

Translation JSON files for English, Hindi, and Assamese. A language selector in the header switches the entire UI. Alert notification content passes through the translation layer before being sent to Twilio. Three languages, one afternoon to wire up.

06 Alert System

Twilio SMS

Free trial provides enough credits for a full demo and then some. The backend exposes **POST /alerts/send-sms**. The handler calls the Twilio REST API with district name, risk level, and the message in the configured language. Response arrives in under 3 seconds. The demo button in the sidebar calls this endpoint and shows a toast on success.

The scheduler also auto-triggers SMS when a district transitions from High to Critical, using registered phone numbers in a config file. This is the automated early warning feature from the problem statement, and it requires zero additional code beyond the manual button handler.

07 Field Reports

POST /reports accepts multipart form data: a photo file, latitude, longitude, and a text description. FastAPI handles multipart natively. The photo is saved to an `/uploads/` directory and the record is written to SQLite with the file path. **GET /reports** returns all reports with a photo URL so the frontend can render the thumbnail in the map marker popup.

Note on Railway deployment: the filesystem is ephemeral, so uploads disappear on redeploy. This does not matter for the demo. The production fix is a 10-line swap from local file save to S3 `put_object`.

08 PWA and Offline Support

The React app is configured as a Progressive Web App using the **Vite PWA Plugin**. This generates a service worker and web manifest automatically. The service worker caches the app shell and the last-fetched district GeoJSON. When opened offline, the map renders with last known risk data and a 'last updated' timestamp.

Field report submissions made offline are queued in **localStorage** with a pending status. A background sync check runs every time the app regains connectivity and flushes the queue to the API. This covers the key demo scenario: a field officer in a low-signal area submits a report that eventually lands on the dashboard.

09 Deployment

Railway (Backend)

Connect the GitHub repo, point Railway at the `backend/` directory, set environment variables (Twilio credentials, IMD API key). Railway detects `requirements.txt` and builds automatically. Every push to main redeploys in under 2 minutes. Free tier is sufficient for hackathon traffic.

Vercel (Frontend)

Connect repo, point at `frontend/`. Vercel handles the Vite build. Set `VITE_API_BASE_URL` to the Railway backend URL as an environment variable. Production URL is stable and shareable immediately.

Wire both to auto-deploy from day one. Deploying continuously means infra issues are caught immediately, not at midnight on the last night.

10 End-to-End Data Flow

Step	What Happens
1. Browser Load	Vercel serves the React app. App calls <code>GET /risk/zones</code> on Railway backend.
2. Map Render	Backend returns NER district GeoJSON enriched with current risk levels. Leaflet renders colored polygons.
3. Scheduler Tick	APScheduler fires every 15 min. Pulls IMD rainfall, runs mock sensor generator, builds feature vectors for all district centroids.
4. ML Inference	Batch <code>predict_proba()</code> call on all districts. Updates in-memory risk dict. Checks threshold crossings.

Step	What Happens
5. Auto Alert	If any district crosses Critical and was not alerted in last hour, Twilio SMS fires to registered contacts automatically.
6. Frontend Sync	React Query refetch (30s interval) picks up updated risk scores. Map recolors the affected district.
7. User Drill-Down	Click district -> sidebar fetches /sensors/live -> Recharts renders 48-hour rainfall + moisture trend.
8. Manual Alert	User clicks Send Alert -> POST /alerts/send-sms -> Twilio call -> SMS in under 5 seconds. Alert logged to SQLite.
9. Field Report	Field officer uploads photo + pin. POST /reports saves to disk + SQLite. Camera marker appears on map immediately.
10. Offline Queue	No connectivity: service worker serves cached app + last GeoJSON. Report queued in localStorage. Flushed when connection returns.

11 Full Tech Stack Summary

Layer	Technology	Reason
Backend API	FastAPI (Python 3.11)	ML lives in same process, async, auto Swagger docs
ML Model	scikit-learn Random Forest	Interpretable, fast inference, handles mixed features
Scheduling	APScheduler	Zero-config in-process background tasks
Database	SQLite + SQLAlchemy	No setup, sufficient for reports and alert logs
Spatial Data	Static GeoJSON files	No PostGIS complexity, sub-millisecond response
Frontend	React + Vite + TypeScript	Fast dev cycle, type safety, hot reload
Maps	React-Leaflet	Best docs, sufficient for polygon + marker rendering
Charts	Recharts	Clean React integration, minimal config
Server State	React Query	Handles polling, caching, and loading states
Multilingual	i18next	Drop-in React library, JSON translation files
SMS Alerts	Twilio Free Tier	Reliable delivery, free credits cover demo
PWA / Offline	Vite PWA Plugin	Service worker and manifest auto-generated
Backend Deploy	Railway	Auto-deploy from GitHub, free tier, 2-min builds
Frontend Deploy	Vercel	Auto-deploy from GitHub, instant CDN